

SMART BICYCLE ANTI-THEFT SYSTEM

ID1050A Engineering Design — Project ID: 18

Team Members:

Sreehari P (102501035)
Haddin John J (132501013)
Gauthamkrishna S (132501012)
Nikhil Karlapudi (112501021)
Daniel Isaac Joy (122501005)

TABLE OF CONTENTS

- Motivation Behind Making the Project
- Defining Our Product
- Demonstration Video of Product
- Mathematical Model
- Circuit Diagram & Hardware Implementation
- Software Implementation
- Deep Learning Prototype: 1D-CNN Tamper Classifier
- Analyse and Improve Phase
- Compare Model vs Test Runs
- Future Scope & Improvements
- Conclusions and Safety Precautions

1. VISION: SECURE & SMART MOBILITY

Replacing traditional, easily-cut cable locks with an electronic intelligence layer.

Our solution bridges the gap between mechanical locks and expensive GPS trackers.

Key Pillars

- **ECONOMIC:** Sub-2,000 cost, accessible to students and commuters
- **ECO-FRIENDLY:** Sustainable alternative to expensive subscription-based trackers
- **GUARANTEES SAFETY:** Real-time tamper alerts and location tracking

2. DEFINE: THE SMART BICYCLE ANTI-THEFT PROTECTION

A compact IoT module built around the ESP32, addressing the rising need for secure personal mobility.

2.1. Features

- **Vibration Detection:** Uses MPU6050 accelerometer to detect tampering
- **Active Deterrence:** Activates a loud buzzer to scare off thieves
- **Real-time Alerts:** Notifies the owner via Classic Bluetooth instantly
- **Telemetry:** Tracks speed and distance using a Hall Effect sensor
- **Health Metrics:** Estimates calories burned based on speed (METs) and rider weight
- **GPS Tracking:** User can track the cycle through the app via Bluetooth communication
- **No Subscriptions:** One-time affordable cost

2.2. Why This Project?

The Problem: Bicycles are evolving, but security hasn't kept up. Cable locks are easily cut. Existing trackers are expensive (2k–6k) or require subscriptions.

The Gap: A lack of a practical, low-cost electronic security layer. Commuters need instant alerts when they are nearby, not just passive tracking.

Our Solution: A sub-2,000 device combining Anti-Theft + GPS + Fitness. Accessible to students and daily commuters.

3. PROJECT TIMELINE (PERT CHART)

The project followed a structured DMAIC (Define, Measure, Analyze, Improve, Control) methodology:

1. **Define Problem** (2 days)
2. **Component Selection** (4 days)
3. **Circuit Design** (2 weeks)
4. **Firmware Coding** (2 weeks)
5. **Firmware Testing** (1 week)
6. **App UI Design** (2 weeks, parallel)
7. **Bluetooth + App Logic** (3 weeks, parallel)
8. **App Testing** (2 weeks)
9. **Enclosure Planning** (1 week)
10. **Fabrication** (2 weeks)
11. **Field Testing** (1 week)
12. **Final Report** (1 week)

4. MEASURE PHASE: MATHEMATICAL MODELS

4.1. Motion & Vibration Detection

We use the MPU6050 accelerometer to calculate the net acceleration vector to detect tampering.

$$V(t) = \sqrt{a_x^2 + a_y^2 + a_z^2}$$

Trigger condition: $V(t) > Threshold \approx 10.5m/s^2$

This threshold is tuned to distinguish genuine tamper attempts from incidental bumps or road vibrations.

4.2. Speed & Calories Estimation

Using Hall Sensor pulses (N) and wheel radius (r). Calorie burn (C) uses Metabolic Equivalent of Task (MET) values.

$$v(t) = \frac{2\pi r N}{t}$$

$$C = MET \times m \times \frac{t}{60}$$

where:

- N = number of hall sensor pulses
- r = wheel radius (user-configurable)
- t = time interval (seconds)
- m = rider weight (user-inputted)
- MET = Metabolic Equivalent of Task (varies with cycling speed)

Reference: <https://blog.nasm.org/metabolic-equivalents-for-weight-loss>

5. CIRCUIT DIAGRAM & HARDWARE IMPLEMENTATION

5.1. System Schematic

Central: ESP32 Microcontroller

Sensors:

- MPU6050 (I2C) — accelerometer/gyroscope
- Neo-6M GPS (UART) — location tracking
- Hall Sensor (GPIO Interrupt) — wheel speed

Power: Li-ion battery (18650 cells) with Buck Converter for stable 5V regulation

Output: Piezo buzzer (GPIO-driven alert)

5.2. Hardware Implementation Details

- **Processing:** ESP32 selected for dual-core performance and built-in Bluetooth/WiFi capabilities
- **Sensing:** MPU6050 for tamper detection, Neo-6M for location logging, Hall Sensor for speed measurement
- **Interface:** Custom acrylic mobile holder fabricated using laser cutting (RDWorks) to mount the phone securely
- **Power Management:** Protected Li-ion cells with step-down converter ensures safe voltage levels for all components

5.3. Physical Assembly

The system evolves from breadboard prototyping to final integration:

1. **Prototyping Phase:** Breadboard-based proof-of-concept with all modules wired individually
2. **Testing Phase:** Modular bring-up of sensor modules (MPU6050, GPS, Hall, BLE) independently
3. **Integration Phase:** Final assembly with proper wire management and enclosure
4. **Field Testing:** Real-world validation on actual bicycles

6. SOFTWARE IMPLEMENTATION

6.1. Core Libraries & Dependencies

- **TinyGPSPlus.h:** Parsing GPS data from Neo-6M module
- **Adafruit_MPU6050.h:** MPU6050 sensor interface
- **Adafruit_Sensor.h:** Common sensor interface library
- **Wire.h:** I2C communication for ESP32
- **BluetoothSerial.h:** Classic Bluetooth SPP for reliable serial communication

6.2. Firmware Architecture & Key Logic Flow

The ESP32 firmware follows a modular structure:

1. **System Initialization:** Sensor setup, Bluetooth pairing
2. **Hall Sensor Module:** Calculates speed and distance via interrupt-driven pulse counting
3. **Motion Detection Module:** Continuous MPU6050 monitoring with magnitude-based threshold
4. **GPS Tracking Module:** UART-based position logging (1 Hz update rate)
5. **Bluetooth Command Handler:** Processes lock/unlock, setweight, and data stream commands
6. **Alert System:** Buzzer activation and push notification triggering
7. **Continuous Loop:** Integrates all modules with 50ms polling interval

6.3. Mobile App (Flutter)

Architecture: Cross-platform Flutter application with Bluetooth Classic SPP interface

Key Screens:

- **HomeScreen:** Bluetooth connection, GPS map display, live status
- **DashboardScreen:** Speed gauge (radial), distance/calories cards, lock/unlock controls, rider weight/wheel config
- **Notifications:** Local push alerts on motion detection

Data Flow: ESP32 streams speed/distance/GPS via UART-over-BT; app parses incoming messages and updates UI in real-time; user commands (LOCK, SETWEIGHT) are sent back to the device.

All code available at: <https://github.com/Haddin08/ProjectSourceCODE.git>

7. DEEP LEARNING PROTOTYPE: 1D-CNN TAMPER CLASSIFIER

7.1. Motivation

The current firmware uses a simple peak-magnitude threshold ($V(t) > 10.5 \text{ m/s}^2$) for tamper detection. While effective, this approach has a fundamental limitation: a single instantaneous threshold cannot distinguish a **brief, high-amplitude bump** (e.g., a dog nudging the bike) from a **sustained, lower-amplitude pattern** (e.g., repetitive sawing or prying at the lock). Both can cross the same magnitude, leading to false positives.

While self-studying deep learning for computer vision through NPTEL, I explored whether a learned classifier operating on short time windows could better capture the *shape* and *duration* of tamper signals rather than relying on peak magnitude alone.

7.2. Approach

Offline 1D-CNN Prototype: I designed a small convolutional neural network (PyTorch) to classify 2-second windows of accelerometer data as either:

- **Normal:** idle jitter, single brief bumps, incidental motion
- **Tamper:** sustained, repetitive vibration patterns

Model Architecture:

- Input: 1 channel (acceleration magnitude), 40 time steps ($20 \text{ Hz} \times 2 \text{ sec}$)
- Layer 1: Conv1d(1→8, kernel=5) + ReLU + MaxPool(2)
- Layer 2: Conv1d(8→16, kernel=5) + ReLU + AdaptiveAvgPool
- Output: Dense(16→2), softmax classification
- **Parameters:** 700 weights — designed for eventual microcontroller deployment

7.3. Training Data & Methodology

IMPORTANT: Synthetic Data Only

I trained entirely on synthetically generated signals, not real accelerometer data from the project. The synthetic signals were designed to encode plausible assumptions:

- **Normal class:** Gaussian noise around resting gravity (9.8 m/s^2), or a single Gaussian spike lasting 1 sec (representing a brief bump)
- **Tamper class:** A sustained sine wave at 2–5 Hz for the full 2-second window (representing repetitive sawing/prying motion)

Why synthetic? Real tamper data doesn't exist yet — validating the hypothesis (that tamper patterns are distinguishably different in duration/frequency content) before investing in field data collection is a lean engineering approach.

Dataset:

- Train: 2,000 synthetic windows (50/50 normal/tamper)
- Validation: 400 synthetic windows
- Epochs: 15, Adam optimizer, cross-entropy loss

7.4. Results (Synthetic Data Validation)

Validation Accuracy: 99.75% (synthetic set)

Class	Precision	Recall	F1-Score	Support
Normal	0.99	1.00	1.00	183
Tamper	1.00	1.00	1.00	217
Weighted Avg	1.00	1.00	1.00	400

Comparison vs. Existing Threshold Rule:

- Agreement with peak-threshold rule: 77.4%
- Cases CNN classified correctly that peak-threshold missed: 113 out of 500 test windows
- Cases peak-threshold got right that CNN missed: 0 out of 500

7.5. What These Numbers Actually Mean

The 99.75% accuracy and 113/500 improvement over the threshold rule apply **only to the synthetic dataset**. They demonstrate that:

1. A small CNN *can* learn to separate "brief transient" from "sustained periodic" signals when that distinction is cleanly present
2. The learned boundary generalizes well within the synthetic distribution

They do **NOT** demonstrate real-world performance, false-positive rate, or robustness on an actual bike, because:

- Synthetic signals omit real-world confounds: potholes, gravel, mounting effects, temperature drift, different lock materials, varied tamper tools
- The hypothesis itself (tamper = sustained 2–5 Hz, normal = brief) has not been validated against real data
- No on-device testing or quantization effects have been evaluated

7.6. Current Status

Prototype Stage: The model weights and training code are in the repository (`ml_prototype/`) for reference and future development.

NOT YET DEPLOYED: This is an offline feasibility study, not yet ported to the ESP32 or integrated into the active firmware. The existing threshold-based detection remains the operational tamper detector.

Next Steps (see Future Scope):

1. Collect real accelerometer logs during controlled bump-vs-tamper tests on an actual bike
2. Retrain the CNN on real data and validate generalization
3. Quantize the model for ESP32 deployment (TensorFlow Lite Micro or similar)
4. A/B test the learned classifier vs. the existing threshold in the field

8. ANALYSE AND IMPROVE PHASE

8.1. Identified Problems & Implemented Solutions

Problem	Solution
BLE Connectivity: Complex pairing and dropped connections with BLE stack	Switched to Bluetooth Classic SPP for reliable serial communication
False Alarms: Single-axis logic triggered alarms on minor tilts	Used vector magnitude $V(t) = \sqrt{a_x^2 + a_y^2 + a_z^2}$ for orientation-invariant detection
Monolithic Code: Hard to debug when all sensors integrated at once	Modular bring-up: developed 4 separate modules (Sensor, Comm, Alarm, Integration) before merging
Threshold Rigidity: Fixed peak magnitude couldn't distinguish signal patterns	Prototyped 1D-CNN classifier to learn duration/frequency distinctions from windowed data

8.2. Engineering Refinements

- **Persistence Counter:** Motion must exceed threshold for 3 consecutive readings before alarm triggers, reducing noise sensitivity
- **Dynamic Calorie Calculation:** METs mapped to speed ranges (4–12 depending on intensity) rather than fixed constant
- **User-Configurable Parameters:** Rider weight and wheel diameter can be set via app, improving accuracy
- **Robust Line Buffering:** Bluetooth stream parsing with explicit newline delimiters handles fragmented packets gracefully

9. MODEL VS. TEST RUNS

Metric	Initial Firmware	Test Threshold	Final Tuned System
Speed Accuracy	Moderate	(4% error), fixed radius assumptions	High accuracy, user-selectable wheel diameter
Tamper Detection	Unstable, too sensitive, frequent false alarms		Robust, tuned threshold (10.5 m/s ²) with persistence counter
Tamper Detection (ML Prototype)	—		99.75% accuracy on synthetic validation set; real-world validation pending
Calorie Estimation	Generic, weight	hardcoded	Realistic, based on dynamic METs and actual user weight
Connectivity	Fragile (BLE), frequent drops	frequent	Stable (Bluetooth Classic SPP) with auto-reconnect logic

10. FUTURE SCOPE AND IMPROVEMENTS

10.1. Hardware Enhancements

1. **Compact Micro-PCB Electronics:** Replace breakout boards with custom PCBs for durability, reduced wiring, and tamper resistance
2. **Frame Integration:** Embed electronics inside the bicycle frame for protection against theft, weather, and shocks
3. **Battery & Solar Optimization:** Use low-power modes and small solar panels for extended operation (weeks/months)

10.2. Connectivity & Tracking

1. **Global Tracking via SIM/GSM:** Enable live GPS tracking, remote tamper alerts, and cloud ride logging beyond Bluetooth range
2. **Mobile App & OTA Updates:** Support firmware updates, theft logs, cloud backups, and multi-bike management

10.3. Machine Learning Integration

1. **Real Sensor Data Collection:** Log accelerometer streams during staged bump-vs-tamper tests on the actual bicycle
2. **CNN Retraining on Real Data:** Validate whether the synthetic-data hypothesis holds; retrain the model with authentic tamper signatures
3. **On-Device Quantization:** Convert the trained PyTorch model to TensorFlow Lite Micro and deploy on the ESP32
4. **Adaptive Learning:** Extend the model to learn location-specific or lock-type-specific patterns over time

10.4. Fitness & Analytics

1. **Enhanced Fitness Metrics:** Add cadence sensors, elevation gain, and ride intensity scoring for richer health insights
2. **Cloud Ride Logging:** Store and analyze ride history for training and health tracking

11. CONCLUSIONS AND SAFETY PRECAUTIONS

11.1. Conclusion

Using the DMAIC methodology, we successfully engineered a low-cost ($\leq 2,000$), reliable Anti-Theft & Fitness module. It solves the critical problem of bicycle theft while democratizing smart fitness metrics in an economical manner.

Key achievements:

- **Real-time tamper detection** via orientation-invariant acceleration magnitude
- **Accurate speed & distance** measurement using Hall-effect sensor interrupts
- **Realistic calorie estimation** based on dynamic METs and user weight
- **Seamless mobile app** with live GPS map, dashboard gauges, and Bluetooth controls
- **Explored ML-based improvements** through a 1D-CNN prototype on synthetic data, establishing a foundation for future real-world refinements

The system is production-ready for the current threshold-based detection and provides a clear roadmap for ML-enhanced detection once real tamper data is collected.

11.2. Safety Precautions

- **Electrical:** Proper insulation to prevent shorts; all high-current paths use fused connections
- **Battery:** Use only protected Li-ion cells (18650 with built-in BMS); never overcharge or short-circuit
- **Mechanical:** Secure all cabling to avoid interference with moving parts (spokes, chain, pedals); use cable ties and heat-shrink tubing
- **Environmental:** Seal enclosure against water ingress; use potted connectors for outdoor durability
- **RF Safety:** Bluetooth operates at 2.4 GHz, well below harmful exposure limits; keep antenna clear of riders